

Project Development Guidelines for Freshers

1. Repository Setup

- Each developer must create a **GitHub repository** in the format:
- <username>/<project-name>
- Repository must be **public or shared with reviewers**.
- Use meaningful project names (avoid generic names like project 1, test-app).

2. Repository Structure

Maintain the following standardized structure:



```
<project-root>
├── docs/                # Design documents, API specs, architecture diagrams
├── api/                 # Backend (.NET Core)
│   ├── src/             # Application source code
│   └── tests/            # Unit and integration tests
├── ui/                  # Frontend (Angular)
│   ├── src/             # Application source code
│   └── tests/            # Unit/UI tests
├── build/ (optional)    # Build scripts (CI helpers, packaging scripts)
├── tools/ (optional)    # Utility scripts (Python / PowerShell / BAT)
└── README.md            # Project overview and setup instructions
```

Expectations

- Strictly follow separation of concerns (UI vs API).
- Avoid mixing frontend and backend logic.
- Ensure test folders exist even if initially minimal.

3. Branching & Commit Strategy

Branching

- Use at least:
 - main → stable code
 - dev → active development
- Feature branches:
feature/<feature-name>

Commit Guidelines

- **Commit frequently** (daily or on logical change).
- Avoid large, monolithic commits.
- Add meaning full commit messages.
"Added user login API"
"Fixed validation bug in signup form"
"Refactored service layer for order module"
- Each commit should:
 - Be **small and meaningful**
 - Represent **one logical change**
 - Be **reviewable**

4. Code Quality Expectations

General

- Follow clean coding principles:
 - Meaningful variable names
 - Modular functions
 - Avoid code duplication

Backend (.NET Core)

- Use layered architecture:
 - Controllers → Services → Repositories
- Implement:
 - Dependency Injection
 - Proper error handling
 - Logging

Frontend (Angular)

- Follow component-based architecture
- Keep:
 - Services for API calls
 - Components for UI logic
- Avoid business logic inside templates

5. Testing Requirements

- Every module should have corresponding tests.

Backend

- Unit tests for services – **Good to have**
- Integration tests for APIs – **Good to have**

Frontend

- Component tests – **Good to have**
- Service tests – **Good to have**
- Minimum expectation:
 - Critical paths must be tested

6. Documentation (docs/ folder)

Must include:

- **Architecture Overview**
- **API Contracts**
- **Data Flow / Sequence Diagrams**
- Any assumptions made

7. README.md (Mandatory)

Your README must clearly explain:

- Project Overview
- Tech Stack Used
- Setup Instructions
- How to Run API
- How to Run UI
- How to Run Tests
- Sample Screenshots (if UI exists)

8. Build & Tooling

build/ (optional)

- Scripts to:
 - Build the project
 - Run locally
 - Package artifacts

tools/ (optional)

- Utility scripts using:
 - Python
 - PowerShell
 - Batch scripts

Examples:

- Data generation scripts
- Automation helpers
- Environment setup

9. Review Criteria

Your project will be evaluated based on:

- Repository structure compliance
- Commit discipline
- Code quality and readability
- Proper separation of concerns
- Test coverage
- Documentation quality
- Ability to run the project end-to-end

10. Non-Negotiables

- No single massive commit
- No missing README
- No mixed frontend/backend code
- No unstructured repository